Monedero — Guion de exposición

Monedero electrónico prepago de circuito cerrado · Gestión de la Calidad del Software · UFG
Expositores: **Steven**, **Bayron** y **David** · Duración total: **9 minutos** + preguntas

Tiempo	Expositor	Tema
0:00 – 3:00	Steven	Qué es, el problema, arquitectura y tecnologías
3:00 – 6:00	Bayron	Reglas del dinero y vistas de la app (titular, caja, administración)
6:00 – 9:00	David	Demostración en vivo con la API, pruebas de calidad y próximos pasos

Antes de empezar (5 minutos antes) — David

- Abrir **Docker Desktop** y esperar "Engine running".
- En `C:\src\monedero`: `dart run tool/up.dart` (base de datos, API y correos).
- En otra consola: `cd apps/app` y `flutter run -d web-server --release --web-port 5000 --web-hostname localhost`.
- Pestañas abiertas: **localhost:5000** (app), **localhost:8025** (Mailpit) y el diagrama de `docs/architecture.md`.
- Una consola grande lista con `dart run tool/demo.dart --pausa` (sin ejecutar todavía).

1. Introducción y arquitectura Steven 0:00 – 3:00

Qué decir:

- "Nuestro proyecto es **Monedero**: una tarjeta prepago de **circuito cerrado**, como la de un supermercado. Se recarga en caja y se gasta solo dentro del comercio."
- "Aunque es académico, lo diseñamos **como si manejara dinero real**: no se puede perder ni duplicar un centavo."
- "Hay tres usuarios: el **titular**, que ve su saldo y su tarjeta; la **caja**, que cobra y recarga; y la **administración**, que emite tarjetas y revisa que todo cuadre."
- "La arquitectura tiene tres piezas: una app **Flutter** para Android y web; una **API en Dart** que aplica las reglas; y **PostgreSQL**, que es la única fuente de verdad del saldo. En local todo corre con **Docker** y cuesta cero."
- "Para producción ya está diseñado: un servidor en Contabo, HTTPS con Caddy, respaldos continuos y, más adelante, tarjetas físicas por NFC. Hoy las tarjetas se simulan con un código QR."

Qué mostrar: el diagrama de `docs/architecture.md` (App → API → PostgreSQL).

2. Las reglas del dinero y las vistas de la app Bayron 3:00 – 6:00

Qué decir:

- "Protegemos el dinero en **tres capas**: la app valida para ayudar al usuario, la API decide, y la base de datos es la **última barrera**."
- "El dinero se guarda en **centavos enteros**, nunca con decimales, para evitar errores de redondeo."

- "Las recargas son solo múltiplos de **$5**; no se permite **sobregiro**; y los cobros mayores de **$25** piden el PIN del titular."
- "Cada operación es **atómica** y lleva una **clave de idempotencia**: si la red falla y la caja reintenta, no se cobra dos veces."
- "Los movimientos **no se pueden modificar ni borrar**, y el saldo siempre debe ser igual a la suma de los movimientos. A eso le llamamos **conciliación**."
- "Cada tarjeta tiene una **firma criptográfica**, así que nadie puede fabricar una tarjeta válida."

Qué mostrar en **localhost:5000** (aclarar: "son vistas de diseño con datos de ejemplo; la lógica real ya funciona en la API"):

- **Inicio:** "API conectada".
- **Titular:** tarjeta virtual con saldo, el QR que se muestra en caja, movimientos y "Reportar pérdida".
- **Caja:** las 3 formas de leer la tarjeta, cobrar, botones de recarga, comprobante y mensajes de error claros.
- **Administración:** resumen con **diferencia de conciliación $0.00**, tabla de tarjetas con sus estados y terminales.

3. Demostración en vivo y calidad David 6:00 – 9:00

Qué decir: "Ahora vamos a usar el sistema de verdad. Este programa hace exactamente las mismas llamadas que hará la app, contra la API y la base de datos reales."

Qué hacer: `dart run tool/demo.dart --pausa` y presionar Enter en cada paso:

Paso	Qué se ve
1. Registrar titular	El código de verificación llega a **Mailpit** (mostrar la pestaña).
2. Emitir tarjeta	Número con dígito Luhn, UID y firma; pasa de PENDING a ACTIVE.
3–4. Recargar $20, cobrar $7.35	Saldo $12.65.
5. Sobregiro de $15	Rechazado: **saldo insuficiente**.
Extra	La misma recarga enviada 5 veces se aplica **una sola vez**; una tarjeta falsificada se rechaza.
6. Reportar pérdida	Segundo código en Mailpit; la tarjeta queda **bloqueada** y la caja ya no puede cobrar.
7. Reponer tarjeta	Tarjeta nueva con el **mismo saldo**; la anterior queda como repuesta.
8. Conciliación	**Diferencia total $0.00**.

Cierre de calidad (≈1 min):

- "Tenemos **pruebas automáticas** que demuestran el diseño, contra PostgreSQL real: **1,000 cobros al mismo tiempo** a la misma tarjeta sin que el saldo quede negativo; la misma operación enviada 5 veces aplicada una sola vez; una reposición que falla a la mitad y se revierte completa; la base que rechaza modificar un movimiento; y la firma falsa rechazada."

> - "Además trabajamos con **calidad de proceso**: Git y GitHub con ramas, Conventional Commits, integración continua, registro de decisiones técnicas (ADR), bitácora de sesiones y una definición de terminado."
> - "Lo siguiente: conectar las vistas a la API, la app en Android, las tarjetas NFC reales y el despliegue. Gracias, ¿preguntas?"

> **Opcional si sobra tiempo:** `cd api` y `dart test` (≈30 s, todo en verde).

Preguntas probables

Pregunta	Respuesta corta
¿Por qué no Firebase?	Necesitamos transacciones con bloqueo, restricciones en la base y conciliación exacta; PostgreSQL lo hace de forma nativa.
¿Qué pasa si se cae el internet?	No se cobra: sin conexión no hay forma segura de evitar el doble gasto. La app solo muestra el último saldo marcado como "no actualizado".
¿Y si roban la tarjeta?	El titular la reporta con un código por correo y queda bloqueada de inmediato; los cobros mayores de $25 piden PIN; la reposición conserva el saldo.
¿Las pantallas ya funcionan?	Hoy son vistas de diseño; la lógica completa ya funciona en la API, como se vio en la demostración. Conectarlas es la siguiente fase.
¿Cuánto cuesta?	La demo cuesta $0. En producción: un VPS barato y almacenamiento de respaldos.

Si algo falla en vivo

- La demo no inicia sesión → `dart run tool/seed.dart`.
- La API no responde → `dart run tool/up.dart`.
- La app web no carga → mostrar las capturas de `docs/img/`.